

Learning & Knowledge Systems

This domain covers software that enables learning, education, and knowledge sharing in any form. Projects can include online learning platforms, AI-powered tutors, peer-to-peer learning systems, and skill development tools. It also includes knowledge management systems, digital libraries, research summarization tools, and platforms for assessment, certification, and credentialing. Whether formal or informal, individual or community-driven, any software that helps people acquire, organize, or apply knowledge fits within this domain.

AI-Powered Knowledge & Learning Platform

Technical Planning & System Architecture Document

1. Introduction

Project Overview

We are building a full-stack AI-powered learning and knowledge-sharing platform that combines:

- Research paper sharing
- Study material management
- AI-assisted learning
- Knowledge graph visualization
- Community discussion forums
- Online learning features
- Assessments and certificates

into a single integrated ecosystem.

The project is designed as a deployable MVP (Minimum Viable Product) capable of supporting approximately 100 active users smoothly.

2. Main Objective of the Project

The primary goal of the platform is to create a centralized environment where:

- Students can learn and share knowledge
- Educators can teach and upload content
- Researchers can share papers and notes
- Users can interact through discussions
- AI can assist users in understanding complex concepts
- Knowledge relationships can be visualized using graphs

This project combines concepts from:

- ResearchGate
- StackOverflow
- Coursera
- Notion

- AI Study Assistants

while maintaining a unified architecture.

3. High-Level System Architecture

Frontend (React + TypeScript)

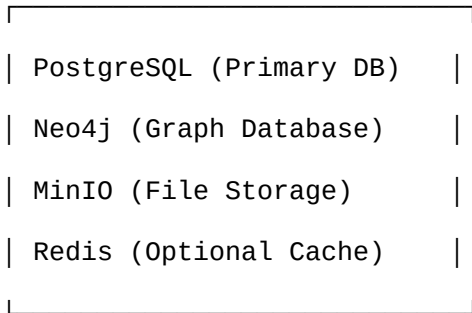


REST APIs / WebSocket



Spring Boot Backend

(Modular Monolith Architecture)



4. Technology Stack

Frontend Technologies

Technology	Purpose
React	Frontend framework
TypeScript	Type safety and maintainability
Vite	Fast frontend build tool
TailwindCSS	UI styling
React Query	API state management
React Router	Routing
Zustand / Redux	Global state management
React Flow /	Graph visualization

D3.js

Backend Technologies

Technology	Purpose
Java 21	Main programming language
Spring Boot	Backend framework
Spring Security	Authentication and authorization
Spring Data JPA	Database operations
Spring AI / LangChain4j	AI integration
WebSocket	Real-time notifications

Database & Infrastructure Technologies

Technology	Purpose
PostgreSQL	Main relational database
Neo4j	Knowledge graph database
MinIO	File storage system
Redis	Cache and optimization
Docker	Containerization
Docker Compose	Multi-service deployment

5. Architecture Decision

Why We Are Using Modular Monolith

We are intentionally NOT using microservices.

Reason:

- Faster development speed
- Easier debugging
- Easier deployment
- Easier communication between modules
- Better for hackathon timeline
- Sufficient for 100 users

All systems will exist inside one Spring Boot project but organized into independent modules.

6. System Breakdown

The project is divided into the following major systems:

1. Authentication & User Management System
2. Knowledge Repository System
3. AI Tutor & Research Assistant System
4. Knowledge Graph System
5. Forum & Q&A System
6. Mini LMS System
7. Assessment System
8. Certificate System
9. Notification System
10. Analytics & Recommendation System

Authentication & User Management System

Purpose

This system handles:

- User registration
- Login/logout
- Security
- User roles
- User profiles
- Access control

Every other system depends on this module.

Without authentication:

- users cannot upload materials,
- courses cannot be managed,
- discussions cannot be tracked,
- AI history cannot be linked,
- ownership cannot be maintained.

Main Responsibilities

The authentication system will:

- identify users
- secure APIs
- manage sessions
- control permissions
- track user activity

Features

1. User Registration

Users should be able to create accounts using:

- email
- username
- password

Optional:

- Google OAuth login

Registration Flow

User submits form

↓

Backend validates input

↓

Password hashed using BCrypt

↓

User stored in PostgreSQL

↓

JWT token generated

2. Login System

Users can:

- login securely
- receive JWT token
- access protected APIs

Login Flow

User enters credentials

↓

Backend validates credentials



JWT access token generated



Frontend stores token



Authenticated requests enabled

3. Role-Based Access Control (RBAC)

Roles

STUDENT

EDUCATOR

ADMIN

Example Permissions

Role	Permissions
Student	Access resources, ask questions
Educator	Create courses, upload materials
Admin	Moderate platform

4. User Profile System

Each user should have:

- profile image
- bio
- uploaded resources
- enrolled courses
- contribution history
- reputation points

5. Reputation System

Users gain points for:

- answering questions
- uploading useful resources
- helping other users
- completing courses

This increases engagement.

Backend Components

Controllers

AuthController

UserController

Services

AuthService

JwtService

UserService

Security Components

JwtFilter

SecurityConfig

PasswordEncoder

Database Tables

users

roles

user_profiles

refresh_tokens

Technologies Needed

Technology	Purpose
Spring Security	API security
JWT	Stateless authentication
PostgreSQL	User data storage
BCrypt	Password hashing

API Endpoints

POST /auth/register

POST /auth/login

POST /auth/logout

GET /users/profile

PUT /users/profile

Knowledge Repository System

Purpose

This is the CORE system of the platform.

This system allows users to:

- upload study materials
- share research papers
- organize knowledge
- search educational content
- interact with uploaded resources

This system behaves like:

"ResearchGate + Google Drive for Learning"

Main Responsibilities

This system handles:

- file uploads
- metadata storage
- resource categorization
- search functionality
- resource visibility
- bookmarks and interactions

Features

1. File Upload System

Supported file types:

- PDF
- DOCX
- PPT

- TXT
- Images (optional)

2. Public / Private Resources

Users should be able to set visibility:

PUBLIC

PRIVATE

UNLISTED

3. Resource Metadata

Each uploaded resource stores:

- title
- description
- tags
- category
- upload date
- uploader information
- file path

4. Search System

Users should search by:

- title
- tag
- topic
- uploader
- category

5. Resource Interactions

Users can:

- bookmark resources
- like resources
- comment on resources
- save collections

6. AI Integration Trigger

After upload:

- AI analysis automatically starts
- concepts extracted
- summaries generated
- graph relationships created

IMPORTANT STORAGE DESIGN

Files should NOT be stored inside PostgreSQL.

Instead:

Use MinIO Object Storage

Why MinIO?

Benefits:

- scalable
- S3 compatible
- Docker friendly
- production-like architecture
- better file management

Upload Flow

User Uploads PDF

↓

Frontend sends multipart request

↓

Backend validates file

↓

File stored in MinIO

↓

Metadata stored in PostgreSQL



AI analysis triggered

Database Tables

resources

resource_tags

bookmarks

resource_comments

resource_likes

Backend Components

Controllers

ResourceController

UploadController

Services

ResourceService

FileStorageService

SearchService

Technologies Needed

Technology	Purpose
MinIO	File storage
Spring	File uploads

Multipart	
PostgreSQL	Metadata storage
JPA	Resource persistence

AI Tutor & Research Assistant System

Purpose

This is the AI layer of the platform.

The purpose of this system is to help users:

- understand difficult concepts
- analyze research papers
- generate study materials
- ask academic questions
- simplify learning

This system provides the major innovation factor.

IMPORTANT DESIGN DECISION

We are NOT training custom AI models.

We are integrating existing AI APIs.

Reason:

- faster development
- lower infrastructure complexity
- better output quality
- easier deployment

Main Features

1. Research Paper Summarization

Users upload papers.

AI generates:

- summary

- key points
- important findings
- extracted concepts

2. Deep Research Mode

This mode performs:

- advanced concept analysis
- prerequisite detection
- detailed explanations
- research insights

Example:

Explain Transformer Architecture deeply

3. Simplified Explanation Mode

Converts:

- complex academic text
into
- beginner-friendly explanation.

4. Quiz Generation

AI creates:

- MCQs
- short questions
- practice tests

based on uploaded material.

5. Flashcard Generation

AI extracts:

- definitions
- formulas
- important concepts

for revision.

6. AI Chat Assistant

Users can ask:

Explain Operating Systems

Summarize this paper

Generate notes on DBMS

AI Pipeline Architecture

PDF Upload

↓

Text Extraction

↓

Chunking

↓

Embedding Generation

↓

AI Processing

↓

Response Generation

PDF Processing

Required tools:

- Apache PDFBox
- Apache Tika

AI Technologies

Technology	Purpose
------------	---------

Gemini API AI generation
OpenAI API Alternative AI
 provider
Spring AI AI integration
LangChain4j AI orchestration

Important Engineering Notes

AI operations should:

- run asynchronously
- avoid blocking requests
- cache repeated responses
- use queues/executor services if needed

Database Tables

ai_requests

ai_summaries

flashcards

generated_quizzes

Knowledge Graph System

Purpose

This system creates relationships between:

- topics
- papers
- skills
- users
- courses

using graph database technology.

Why We Need Neo4j

Normal SQL databases are good for:

- structured relational data

But graph databases are better for:

- connected knowledge
- recommendations
- prerequisite mapping
- relationship traversal

Main Features

1. Topic Relationship Mapping

Example:

Machine Learning

↓ requires

Statistics

2. Related Resource Recommendation

Example:

Users studying Spring Boot

also study Docker

3. Learning Roadmaps

Visual prerequisite chains.

4. Skill Graph

Tracks:

- user interests
- learning progression
- expertise domains

5. Knowledge Visualization

Users should visually see:

- connected concepts
- topic networks
- dependency graphs

Neo4j Data Model Example

(User) - [:UPLOADED] -> (Paper)

(Paper) - [:RELATED_TO] -> (Topic)

(Topic) - [:REQUIRES] -> (Skill)

Frontend Visualization

Recommended:

- React Flow

- D3.js

Important Design Note

Neo4j is:

- secondary database
- recommendation layer
- relationship engine

PostgreSQL remains:

- primary source of truth

Technologies Needed

Technology	Purpose
Neo4j	Graph database
Spring Data Neo4j	Neo4j integration
React Flow	Graph UI

Forum & Q&A System

Purpose

This system enables:

- community discussion
- doubt solving
- peer learning

Similar to:

- StackOverflow

Features

Core Features

- ask question
- answer question
- upvote answers
- accepted answers
- tagging system
- markdown support

Additional Features

- search questions
- filter by tags
- user reputation
- trending discussions

Database Tables

questions

answers

votes

tags

question_tags

Backend Components

ForumController

QuestionService

AnswerService

VoteService

Technologies Needed

Technology	Purpose
PostgreSQL	Discussion storage
Markdown Editor	Rich question formatting
Spring Boot	Backend APIs

Mini LMS System

Purpose

This system allows educators to:

- create courses
- upload lessons
- organize learning material
- teach students

IMPORTANT

This should remain:

- lightweight
- minimal
- easy to maintain

We are NOT building a full Coursera clone.

Features

Course Management

Educators can:

- create courses
- edit courses
- upload lessons
- manage students

Student Features

Students can:

- enroll in courses
- track progress

- complete lessons

Lesson Types

Supported:

- video links
- PDFs
- notes
- text lessons

Database Tables

courses

lessons

enrollments

progress

Assessment System

Purpose

Allows users to:

- test knowledge
- complete quizzes
- practice concepts

Features

- MCQ quizzes
- timers
- auto evaluation
- score calculation
- result storage

Quiz Flow

User starts quiz



Questions fetched



User submits answers



Backend evaluates score



Results stored

Database Tables

quizzes

quiz_questions

quiz_submissions

scores

Certificate System

Purpose

Generate completion certificates.

Connected to:

- LMS
- assessments
- achievements

Features

- PDF generation
- unique certificate ID
- downloadable certificate
- verification support

Technologies Needed

Technology	Purpose
PDF libraries	Certificate generation
Spring Boot	Backend generation

Notification System

Purpose

Keeps users updated about platform activities.

Features

- AI task completed
- new answers
- course updates
- assignment reminders
- new recommendations

Implementation Strategy

Phase 1:

- simple polling

Phase 2 (optional):

- WebSocket real-time notifications

Database Tables

notifications

notification_reads

Analytics & Recommendation System

Purpose

Tracks:

- user activity
- learning progress
- platform engagement
- recommendations

Features

User Analytics

- uploaded resources
- completed quizzes
- learning streaks

Platform Analytics

- trending topics
- active users
- popular resources

Recommendation Engine

Uses:

- Neo4j relationships
- user interactions
- topic similarity

Deployment Architecture

Frontend Deployment

Deploy React frontend on:

- Vercel

Backend Deployment

Deploy Spring Boot backend on:

- Railway
- Render

PostgreSQL Hosting

- Neon PostgreSQL
- Railway PostgreSQL

Neo4j Hosting

- Neo4j AuraDB

Docker Setup

All services should run using Docker Compose.

Docker Services

frontend

backend

postgres

neo4j

minio

redis(optional)

Development Strategy

Priority Order

Phase 1 (Foundation)

1. Authentication System
2. Database Setup
3. Docker Setup
4. Backend Architecture

Phase 2 (Core Features)

5. Knowledge Repository
6. AI Assistant
7. Neo4j Graph System

Phase 3 (Community Features)

8. Forum System
9. Mini LMS
10. Assessments

Phase 4 (Finalization)

11. Certificates
12. Notifications
13. UI Polish
14. Deployment
15. Testing

Final Important Engineering Notes

1. Focus on Integration

The project should feel:

- connected
- smooth
- unified

The systems should work together naturally.

2. Avoid Overengineering

Do NOT:

- build unnecessary microservices
- overcomplicate infrastructure
- add too many unfinished features

3. Prioritize Stability

The platform should:

- deploy successfully
- run smoothly
- avoid crashes
- support 100 users reliably

4. Build MVP First

Complete core systems before advanced features.

Core systems:

Auth

Knowledge Repository

AI Assistant

Neo4j Graph

Forum

Deployment

Conclusion

This project combines:

- Full-stack engineering
- AI integration
- Knowledge systems
- Graph databases
- Community learning
- DevOps deployment

into one scalable educational ecosystem.

The primary goal is to build a clean, functional, deployable platform where all systems integrate properly and demonstrate strong engineering design.